UADER - FCyT | Ingeniería de Software II

Guía Maestra de Estudio y Defensa: TPFI

Proyecto: TPFI-PROJECT (Arquitectura Distribuida con Patrones GoF y AWS DynamoDB)

Autores del Repositorio: Camila Viñeg, Tadeo Rochas, Abril Poggio | **Versión:** 0.5.0

> **Objetivo de esta guía:** Proporcionar una comprensión profunda de la arquitectura, patrones de diseño, decisiones técnicas, flujo de ejecución, pruebas automatizadas y preguntas de coloquio para defender el Trabajo Práctico con éxito ante la cátedra.

1. Visión General del Sistema y Requerimientos

El sistema es una **aplicación cliente-servidor distribuida sobre TCP** que gestiona información corporativa de sedes institucionales en una base de datos NoSQL (**AWS DynamoDB**).

Programas requeridos por la consigna:

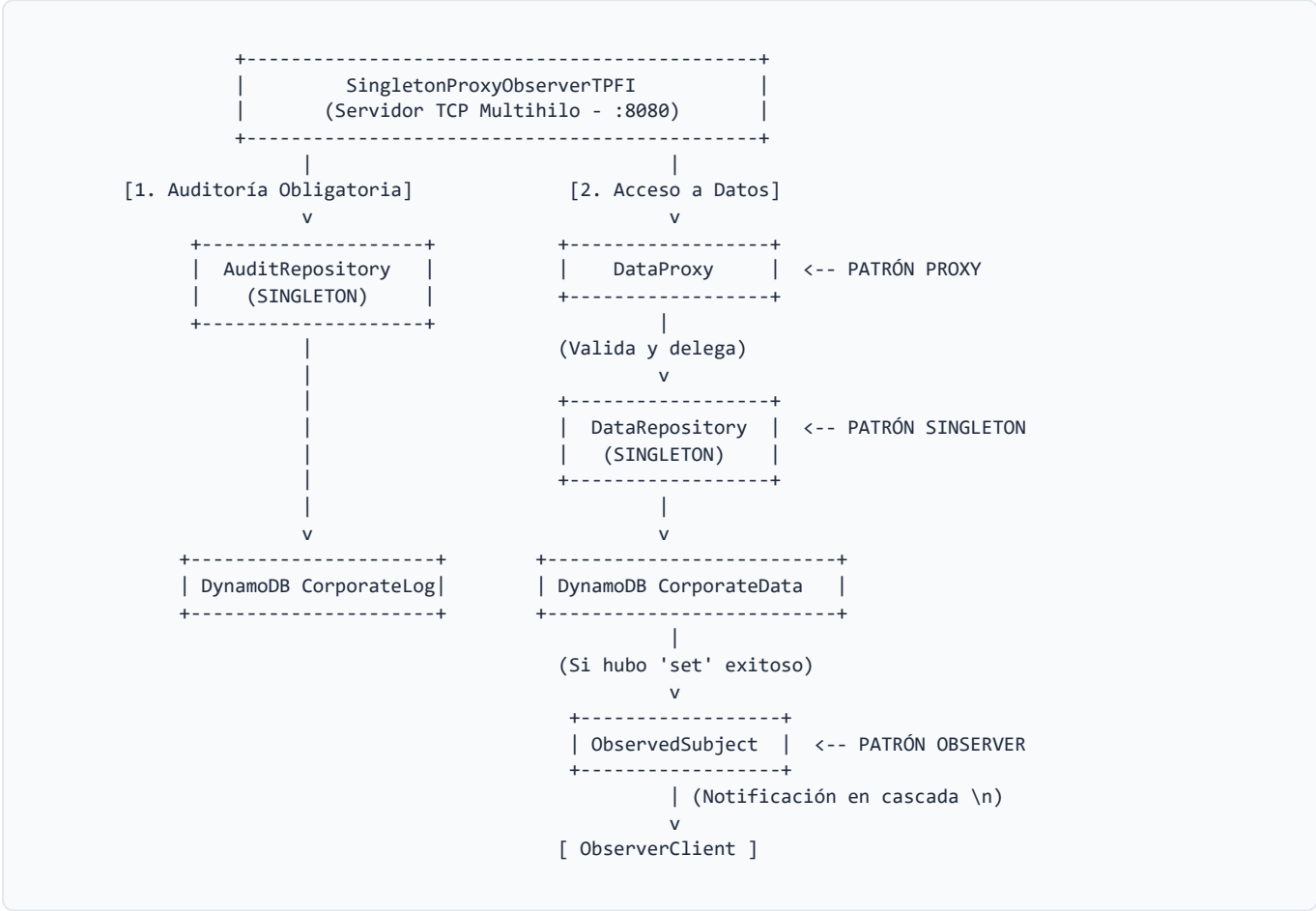
Programa (CLI)	Archivo fuente	Rol y Responsabilidad
`singletonproxyobserver`	`src/tpfi/singletonproxyobserver.py`	Servidor TCP multihilo (`SingletonProxyObserverTPFI`). Recibe conexiones concurrentes, audita cada petición en `CorporateLog`, intercepta y valida mediante Proxy, ejecuta la persistencia física vía Singleton y emite notificaciones a observadores.
`singletonclient`	`src/tpfi/singletonclient.py`	Cliente transaccional puntual (`SingletonClient`). Lee una petición JSON (`-i input.json`), consulta o actualiza la tabla y emite la respuesta (`-o output.json` o consola). Soporta tanto el formato propio como el formato con mayúsculas a nivel raíz de la consigna.
`observerclient`	`src/tpfi/observerclient.py`	Cliente suscriptor reactivo (`ObserverClient`). Se conecta, envía su UUID de CPU (`subscribe`) y mantiene el socket abierto para recibir actualizaciones en tiempo real cada vez que un registro cambia. Cuenta con reconexión automática ante caídas de red.

Las 2 Tablas de AWS DynamoDB:

- **`CorporateData` :** Almacena los registros de sedes institucionales.
  - **Clave primaria (Partition Key):** `id` (texto no vacío, identificador único de la sede).
  - **Campos del tuple corporativo:** `cp` , `CUIT` , `domicilio` , `idreq` , `idSeq` , `localidad` , `provincia` , `sede` , `seqID` , `telefono` , `web` .

- `CorporateLog` **:** Pista de **auditoría obligatoria** por requerimiento de seguridad.
    - **Clave primaria:** `id` (almacena el `session_id` generado autoritativamente por el servidor).
    - **Atributos registrados:** `action` (`get` , `set` , `list` , `subscribe`), `client_uuid` (identificador de CPU del solicitante), `session_id` , `timestamp` (ISO 8601 UTC), `key` (el `id` afectado o `*` si aplica a toda la tabla), y `cpu_info` (nodo, SO, arquitectura, procesador y UUID vía `uuid.getnode()`).

2. Arquitectura y los 4 Patrones de Diseño (GoF) en Detalle



1. Patrón Singleton (`tpfi.patterns.singleton` y `tpfi.repository`)

- **Problema que resuelve:** En un servidor multihilo, abrir clientes de AWS (`boto3.resource`) en cada petición agota los sockets, satura file descriptors y añade latencia. Se requiere un punto de acceso único y compartido a la base física.
- **Implementación:** Metaclase `SingletonABCMeta` (hereda de `abc.ABCMeta`). Intercepta la creación mediante `__call__` aplicando **Double-Checked Locking** con `threading.Lock` para garantizar seguridad multihilo sin bloquear lecturas concurrentes.
- **Uso:** Se aplica a `DataRepository` y a `AuditRepository` como Singletons independientes.

2. Patrón Proxy (`tpfi.patterns.proxy` y `tpfi.proxy`)

- **Problema que resuelve:** El servidor jamás debe comunicarse directamente con DynamoDB sin control. El Proxy actúa como intermediario de protección y validación de reglas de negocio.
- **Los 5 Pasos de Validación en `DataProxy.set()` :**
    1. **Validación de clave:** Comprueba que `id` sea texto no vacío.
    2. **Validación de estructura:** Asegura que `values` sea un objeto JSON.
    3. **Protección de clave reservada:** Bloquea cualquier intento de modificar la clave primaria `id` dentro de `data` .
    4. **Validación estricta de esquema:** Rechaza cualquier atributo ajeno a `CORPORATE_FIELDS` .

5. **Delegación atómica:** Delega en `repository.merge()`.

3. Patrón Observer (`tpfi.patterns.observer`)

- **Problema que resuelve:** Notificar en tiempo real a clientes suscriptos cada vez que un registro corporativo se modifique (`set`), eliminando la necesidad de polling.
- **Snapshot Isolation:** El sujeto (`ObservedSubject`) clona la lista de observadores bajo lock y libera el cerrojo antes de transmitir por red, evitando deadlocks o cuellos de botella con clientes lentos.
- **Poda Automática (Auto-pruning):** Si un socket falla (`SocketDeliveryError`), el sujeto lo desuscribe automáticamente liberando los recursos del sistema.
- **Soporte multiconexión:** Permite múltiples observadores concurrentes desde una misma máquina (mismo CPU UUID).

4. Patrón Iterator (`tpfi.patterns.iterator` y `tpfi.collection`)

- **Problema que resuelve:** DynamoDB limita las lecturas de `scan()` a **1 MB por llamada**. Si una tabla supera ese tamaño, DynamoDB trunca la respuesta y devuelve `LastEvaluatedKey`. Sin iterador, los listados quedan incompletos.
- **Mapeo GoF:** `AggregateMixin` (Aggregate), `CorporateDataCollection` (ConcreteAggregate), `IteratorProtocol` (Iterator) y `RecordIterator` (ConcreteIterator).
- **Evaluación Perezosa (Lazy):** `RecordIterator` solicita páginas a DynamoDB recién cuando se agota la anterior. Permite además aplicar filtros y ordenación en el flujo. Al finalizar, su método `close()` cierra el generador subyacente.

3. Protocolo de Comunicación y Red (TCP Stream Framing)

TCP es un protocolo orientado a flujo continuo de bytes (*stream*). Dos mensajes consecutivos pueden llegar juntos o un mensaje puede llegar fragmentado.

> **Encuadre (Line-delimited JSON):** Cada mensaje viaja serializado como una línea JSON terminada estrictamente con el delimitador `\n`.

La clase `JsonSocket` (`src/tpfi/protocol.py`) gestiona un búfer interno, detecta `\n` y devuelve mensajes íntegros. Además, impone un límite de **1 MiB (`MAX_MESSAGE_SIZE`)** como protección contra ataques de denegación de servicio (DoS por consumo de memoria).

Esquema de los 4 Mensajes del Protocolo:

```
// 1. Requerimiento del Cliente (Request)
{
  "type": "request",
  "action": "get" | "set" | "list" | "subscribe",
  "session_id": "uuid-opcional",
  "client_uuid": "uuid-cpu",
  "id": "UADER-FCyT-IS2",
  "data": { "telefono": "03442 43-1442" }
}

// 2. Respuesta Exitosa del Servidor (Response OK)
{
  "type": "response",
  "status": "ok",
  "session_id": "uuid4-generado-por-servidor",
  "data": { "id": "UADER-FCyT-IS2", "sede": "FCyT Central", ... }
}

// 3. Respuesta de Error del Servidor (Response Error)
{
  "type": "response",
  "status": "error",
  "session_id": "uuid4-generado-por-servidor",
  "error": "No se admiten los campos reservados: id"
}

// 4. Notificación en Cascada para Observadores (Notification)
{
  "type": "notification",
  "action": "set",
  "session_id": "uuid4-generado-por-servidor",
  "data": { "id": "UADER-FCyT-IS2", "telefono": "03442 43-1442", ... }
}
```

4. Auditoría, Concurrencia y Persistencia Atómica

- **Autoridad del `session_id`:** El servidor genera un `uuid.uuid4()` autoritativo en cada petición entrante. Evita que un cliente malicioso sobrescriba registros de auditoría previos en `CorporateLog` o suplante sesiones.
- **Actualización Atómica (`DataRepository.merge`):** Se delega la fusión directamente en DynamoDB mediante una única operación `update_item` con expresiones `SET` e `if_not_exists`. Esto elimina la condición de carrera (*race condition*) que ocurriría si se hiciera un `get` seguido de un `put` en Python.
- **Detección de doble bind en Windows:** Se desactiva `allow_reuse_address` para `sys.platform == 'win32'`, impidiendo que dos servidores escuchen simultáneamente en el mismo puerto.

5. Estrategia de Pruebas Automatizadas (195 Tests - 95.03% Cobertura)

- **Pruebas Unitarias:** Dobles de prueba para aislar la red TCP y las llamadas a DynamoDB.
- **Suite de Aceptación (`tests/test_acceptance.py`):** Cubre los 5 escenarios obligatorios: camino feliz con impacto en ambas tablas, argumentos CLI y JSON corruptos, peticiones sin datos mínimos, comportamiento ante servidor caído y rechazo de doble bind.
- **Pruebas de Propiedades con Hypothesis (`tests/test_hypothesis.py`):** Fuzzing del protocolo con secuencias arbitrarias de bytes e ida y vuelta de serialización JSON.
- **Control Estático:** `ruff check .` (0 errores), `black --check .` (100% homogéneo), `mypy --strict` (tipado estricto verificado) y `bandit` (0 vulnerabilidades).

6. Protocolo de Demostración en Vivo para la Presentación

> **Demostración Oficial en 3 Terminales:** Ejecutar los comandos en el orden indicado para ilustrar la interacción en red y la notificación en tiempo real.

Terminal 1: Servidor de Aplicaciones

```
python -m tpfi.singletonproxyobserver -p 8080 -v
```

Terminal 2: Observador Reactivo

```
python -m tpfi.observerclient -s 127.0.0.1 -p 8080 -v
```

Terminal 3: Cliente de Transacciones

```
# 1. Consulta por clave (Lectura intermediada por Proxy)
python -m tpfi.singletonclient -s 127.0.0.1 -p 8080 -i "samples/input_get.json" -v

# 2. Alta del registro demo TPFI-DEMO-2026 (La Terminal 2 recibe la notificación en cascada en tiempo
python -m tpfi.singletonclient -s 127.0.0.1 -p 8080 -i "samples/input_set.json" -v

# 3. Listado completo (Paginación transparente con el patrón Iterator)
python -m tpfi.singletonclient -s 127.0.0.1 -p 8080 -i "samples/input_list.json" -v

# 4. Intento inválido (El Proxy rechaza alterar la clave primaria reservada)
python -m tpfi.singletonclient -s 127.0.0.1 -p 8080 -i "samples/input_invalido.json" -v
```

Plan de contingencia offline: Si no hay conexión a AWS en el aula, ejecutar `python demo_runner.py` para mostrar el ciclo completo con repositorios en memoria.

7. Banco Maestro de Preguntas para el Coloquio Oral

P1: ¿Por qué usaron una metaclase para el Singleton en lugar de un decorador o get_instance()?

R: Porque una metaclase (`SingletonABCMeta`) gobierna la instanciación a nivel de lenguaje interceptando `__call__` . Permite que el resto del sistema instancie la clase de forma idiomática con `DataRepository()` sin acoplarse a funciones externas, admite herencia limpia de `abc.ABC` y se sincroniza con *Double-Checked Locking*.

P2: ¿Por qué la validación reside en el Proxy y no dentro del Repositorio?

R: Por el Principio de Responsabilidad Única (SRP). `DataRepository` solo se encarga del acceso físico a DynamoDB. `DataProxy` aplica las reglas de negocio (campos obligatorios, protección de claves reservadas y rechazo de campos ajenos), permitiendo cambiar de motor de base de datos o usar dobles de prueba sin alterar la lógica de validación.

P3: ¿Qué sucede si dos clientes envían un set concurrente sobre el mismo registro?

R: Si hiciéramos un `get` previo en Python y luego un `put` , habría una condición de carrera (*race condition*). En nuestro diseño, `DataRepository.merge()` delega la operación en DynamoDB mediante una llamada atómica a `update_item` con expresiones de actualización, garantizando atomicidad a nivel de registro.

P4: ¿Por qué el session_id lo crea el servidor y no el cliente?

R: Por seguridad e integridad de la auditoría. Si el cliente generara el identificador de sesión, podría inventar identificadores repetidos para sobrescribir entradas de auditoría previas en `CorporateLog` o suplantar sesiones ajenas. El servidor actúa como autoridad certificante generando un `uuid.uuid4()` único.

P5: ¿Por qué fue indispensable el patrón Iterator en la versión 0.5.0?

R: DynamoDB limita las lecturas de `scan` a 1 MB por llamada. Si la tabla supera ese límite, la respuesta se trunca y devuelve `LastEvaluatedKey`. El iterador (`RecordIterator`) resuelve la paginación transparente multietapa de forma perezosa (lazy evaluation), solicitando páginas sucesivas a AWS recién cuando el consumidor agota la actual sin saturar la memoria RAM.

P6: ¿Cómo se maneja la desconexión abrupta de un cliente observador?

R: En el servidor, al intentar enviar una notificación o detectar el cierre del canal, se captura la excepción de socket (`SocketDeliveryError`). El sujeto invoca automáticamente `unsubscribe()` y libera el descriptor (poda automática). En el cliente, `ObserverClient` captura el error y activa un bucle de reconexión periódica.

P7: ¿Cómo se protege el servidor contra ataques de denegación de servicio por memoria en sockets?

R: En el protocolo (`JsonSocket`), se impone un límite máximo de tamaño de mensaje de 1 MiB (`MAX_MESSAGE_SIZE`). Si un emisor envía un flujo continuo de bytes sin el delimitador `\n`, el socket interrumpe la lectura y aborta la conexión antes de consumir la memoria del servidor.

P8: ¿Por qué el patrón Observer utiliza Snapshot Isolation al momento de notificar?

R: Porque transmitir datos por sockets TCP hacia múltiples clientes es una operación lenta sujeta a latencias de red. Si mantuviéramos el cerrojo adquirido durante las llamadas a `sendall`, un observador con conexión lenta bloquearía a todos los hilos del servidor. Al clonar la lista de observadores bajo lock y liberarlo de inmediato, la difusión se ejecuta sin frenar el servidor.